

CSE 222 ASSIGNMENT 4 REPORT

Purpose:

This assignment asked us to create a Java program for a planetary system. We needed to:

- Build a tree structure with a star, planets, and moons.
- Add sensor data like temperature, pressure, humidity, and radiation for each body.
- Implement commands to create the system, add planets/moons, find radiation anomalies, show paths, and print reports.
- Write Javadoc comments for all classes and methods to create clear documentation.
- Generate HTML documentation using the Javadoc tool.

The goal was to manage celestial bodies, handle errors (like no humidity for stars), and make the code easy to understand with proper documentation.

Classes Used to Achieve the Goal

We created three Java classes to build the planetary system. Here's what they do, including the values they take and the types they create:

1. PlanetarySystem Class

- **Purpose:** Manages the whole system.
- **What it does:** Handles commands (e.g., create star, add planet, print reports) and builds the tree structure.
- **Values taken:**
 - starName (String), temperature (double), pressure (double), humidity (double), radiation (double) for creating a star.
 - Similar values for adding planets/moons, plus parentName (String).
 - threshold (double) for finding radiation anomalies.
- **Types created:**
 - Node (root of the tree).
 - List<Node> for radiation anomalies.

- `Stack<String>` for paths to a node.
- **Key tasks:** Processes data like radiation anomalies and generates reports.

2. Node Class

- **Purpose:** Represents a star, planet, or moon.
- **What it does:** Stores details and connects bodies in the tree.
- **Values taken:**
 - `name (String)`, `type (String: "Star", "Planet", or "Moon")`, `sensorData (SensorData)` for construction.
 - `child (Node)` for adding children.
- **Types created:**
 - `String` for name and type.
 - `SensorData` for sensor info.
 - `List<Node>` for children (e.g., moons for a planet).
- **Key tasks:** Helps find paths and manage tree connections.

3. SensorData Class

- **Purpose:** Holds sensor data for each body.
- **What it does:** Organizes values and formats them for reports.
- **Values taken:**
 - `temperature (double)`, `pressure (double)`, `humidity (double)`, `radiation (double)` for construction.
- **Types created:**
 - `double` for each sensor value (`temperature`, `pressure`, `humidity`, `radiation`).
 - `String` for formatted output (via `toString`).
- **Key tasks:** Provides data for checks (e.g., radiation) and reports.

How They Work Together

- PlanetarySystem uses Node to create the tree (e.g., Sun → Earth → Moon).
- Each Node has a SensorData to store its values (e.g., temperature, radiation).
- Javadoc comments in all classes make the code clear and generate HTML documentation.

Error Handling in the Program

Our program catches errors to keep it stable and show clear messages. Here are the main errors we handle:

1. Existing System Error

- Trying to create a new system when one exists.
- Error: `IllegalStateException` ("A planetary system already exists!").
- Why: Prevents overwriting the system.

2. Humidity for Stars Error

- Setting humidity for a star (not allowed).
- Error: `IllegalArgumentException` ("Stars cannot have humidity!").
- Why: Stars can't have humidity in real life.

3. Negative Values Error

- Using negative sensor values (temperature, pressure, etc.).
- Error: Prints "Invalid sensor data: Values cannot be negative!" or throws `IllegalArgumentException` in `SensorData`.
- Why: Negative values are invalid.

4. Duplicate Name Error

- Adding a planet/moon with an existing name.
- Error: Prints "A node with name [name] already exists!".
- Why: Ensures unique names.

5. Invalid Parent Error

- Adding a planet to a moon or a moon to a non-planet.

- **Error:** Prints "Cannot add a planet to a moon!" or "Parent [name] not found or not a planet!".
- **Why:** Follows tree structure rules.

6. Invalid Command Error

- **Wrong commands or bad inputs** (e.g., text for numbers).
- **Error:** `NumberFormatException` ("Invalid number format...") or `ArrayIndexOutOfBoundsException` ("Insufficient arguments...").
- **Why:** Guides users to fix inputs.

Why It Matters

- **Error checks prevent crashes.**
- **Messages explain what's wrong.**
- **Javadoc (@throws) documents errors clearly.**

Error Handling in the Program

Our program checks for errors to make sure it runs smoothly and gives clear messages when something goes wrong. Below are the main errors we handle and how we manage them:

1. Existing System Error

- **What happens:** If you try to create a new planetary system when one already exists, the program stops it.
- **Error type:** `IllegalStateException`.
- **How we handle it:** The `createPlanetSystem` method checks if the root node exists. If it does, it throws this error with the message: "A planetary system already exists!"
- **Why:** This prevents overwriting an existing system.

2. Humidity for Stars Error

- **What happens:** Stars cannot have humidity, so if you try to set humidity for a star, the program catches it.

- **Error type:** IllegalArgumentException.
- **How we handle it:** The createPlanetSystem method checks if humidity is not zero. If it's not, it throws this error with the message: "Stars cannot have humidity!"
- **Why:** This ensures stars follow real-world rules.

3. Negative Values Error

- **What happens:** Sensor data (temperature, pressure, humidity, radiation) cannot be negative.
- **Error type:** None thrown; prints a message.
- **How we handle it:** In addPlanet and addSatellite, we check if any value is negative. If so, it prints: "Invalid sensor data: Values cannot be negative!" and skips the action. In SensorData, we throw IllegalArgumentException for negative values during construction.
- **Why:** Negative values don't make sense for sensor data.

4. Duplicate Name Error

- **What happens:** You can't add a planet or moon with a name that's already used.
- **Error type:** None thrown; prints a message.
- **How we handle it:** The findNode method checks if the name exists. If it does, addPlanet or addSatellite prints: "A node with name [name] already exists!" and stops.
- **Why:** This keeps every node's name unique.

5. Invalid Parent Error

- **What happens:** You can't add a planet to a moon or a moon to a non-planet.

- **Error type:** None thrown; prints a message.
- **How we handle it:**
 - For planets, addPlanet checks if the parent is a moon. If so, it prints: "Cannot add a planet to a moon!"
 - For moons, addSatellite checks if the parent is a planet. If not, it prints: "Parent [name] not found or not a planet!"
- **Why:** This follows the tree structure rules (e.g., moons only under planets).

6. Invalid Command or Input Error

- **What happens:** Wrong commands or bad number formats (e.g., text instead of a number) cause issues.
- **Error types:** NumberFormatException, ArrayIndexOutOfBoundsException.
- **How we handle it:** The executeCommand method catches these errors and prints:
 - "Invalid number format in command: [command]" for bad numbers.
 - "Insufficient arguments for command: [command]" for missing inputs.
- **Why:** This helps users fix their commands without crashing the program.

How Error Handling Helps

- These checks stop the program from breaking when bad inputs happen.
- Clear messages tell the user what went wrong (e.g., "Parent not found").
- Javadoc comments explain these errors (like @throws) so others understand how the code handles issues.

Input and Output in the Program

Our program uses a terminal interface to take inputs from the user and show outputs. Below, we explain how it works:

Inputs

- The program reads commands from the user through the terminal.
- Each command is a line of text with specific parts, like names and numbers.

- Supported commands include:
 - create planetSystem [starName] [temperature] [pressure] [humidity] [radiation]
Example: create planetSystem Sun 5500 10000 0 0.1
 - addPlanet [planetName] [parentName] [temperature] [pressure] [humidity] [radiation]
Example: addPlanet Earth Sun 288 101325 60 0.0001
 - addSatellite [satelliteName] [parentName] [temperature] [pressure] [humidity] [radiation]
Example: addSatellite Moon Earth 173 0 0 0.00005
 - findRadiationAnomalies [threshold]
Example: findRadiationAnomalies 0.01
 - getPathTo [nodeName]
Example: getPathTo Moon
 - printMissionReport or printMissionReport [nodeName]
Example: printMissionReport or printMissionReport Earth
 - exit to stop the program.
- Users enter commands one by one, and an empty line or exit stops input.

Outputs

- The program prints messages to the terminal to show results or errors.

- Examples of mini test;

Inputs:

```

java PlanetarySystem
Planetary System Terminal UI.
Paste your commands below (end with an empty line or 'exit' to finish):
Available commands: create planetSystem, addPlanet, addSatellite, findRadiationAnomalies, getPathTo, printMissionReport
create planetSystem Sun 5500 10000 0 0.1
addPlanet Earth Sun 288 101325 60 0.0001
addSatellite Moon Earth 173 0 0 0.00005
addPlanet Mars Sun 210 600 0 0.0002
findRadiationAnomalies 0.01
getPathTo Moon
printMissionReport
printMissionReport Earth
addPlanet Earth Sun 300 100000 50 0.1
addPlanet Jupiter Moon 150 100000 10 0.05
create planetSystem StarX 6000 20000 10 0.2
addPlanet Venus Sun -100 90000 20 0.03
exit

```

Outputs:

```

Executing: create planetSystem Sun 5500 10000 0 0.1
Planetary system created with star: Sun

Executing: addPlanet Earth Sun 288 101325 60 0.0001
Planet Earth added under Sun

Executing: addSatellite Moon Earth 173 0 0 0.00005
Satellite Moon added under Earth

Executing: addPlanet Mars Sun 210 600 0 0.0002
Planet Mars added under Sun

Executing: findRadiationAnomalies 0.01
Radiation anomalies (>0.01 Sv):
Star Sun: 0.1

Executing: getPathTo Moon
Path to Moon: [Sun, Earth, Moon]

Executing: printMissionReport
Star Sun: 5500.0 Kelvin, 10000.0 Pascals, 0.0% Humidity, 0.1 Sieverts
  Planet Earth: 288.0 Kelvin, 101325.0 Pascals, 60.0% Humidity, 1.0E-4 Sieverts
    Moon Moon: 173.0 Kelvin, 0.0 Pascals, 0.0% Humidity, 5.0E-5 Sieverts
  Planet Mars: 210.0 Kelvin, 600.0 Pascals, 0.0% Humidity, 2.0E-4 Sieverts

Executing: printMissionReport Earth
Planet Earth: 288.0 Kelvin, 101325.0 Pascals, 60.0% Humidity, 1.0E-4 Sieverts

Executing: addPlanet Earth Sun 300 100000 50 0.1
A node with name Earth already exists!

Executing: addPlanet Jupiter Moon 150 100000 10 0.05
Cannot add a planet to a moon!

Executing: create planetSystem StarX 6000 20000 10 0.2
A planetary system already exists!

Executing: addPlanet Venus Sun -100 90000 20 0.03
Invalid sensor data: Values cannot be negative!
Processing complete.

```

UML Diagram of System;



Summary

This project created a Java program to manage a planetary system as a tree structure. The goal was to build a system with a star, planets, and moons, handle sensor data (temperature, pressure, humidity, radiation), and generate reports. We achieved this using three classes:

- **PlanetarySystem** controls the system and processes commands.
- **Node** represents celestial bodies (star, planet, moon) and their connections.
- **SensorData** stores sensor values for each body.

The program handles errors like negative values, duplicate names, or invalid commands, showing clear messages to the user. It takes inputs through a terminal (e.g., create planetSystem, addPlanet) and produces outputs like mission reports or paths (e.g., Sun → Earth → Moon). We added Javadoc comments to explain all classes and methods, generating HTML documentation for clarity. A UML diagram shows how the classes connect, making the structure easy to understand.

Overall, the program is modular, user-friendly, and well-documented, meeting all the assignment's requirements.

Acknowledgments

I would like to thank my instructor for providing clear guidelines for this assignment. Their support helped me understand Java programming and documentation better. This project was completed by Fatih Emre Oğan.